

MA 232 Course Project 1: The Leontief Input-Output Model

Linear Algebra & Macroeconomic Interdependence

Group members: Ariel Miranda, Vincent Wang, Tia Hussain

*All data is loaded from `data.py`, which is transcribed from the assignment PDF. Every number in this report is computed by the code below; none is typed in by hand. Units are **billions of dollars**. Production levels are reported to 3 decimals (the nearest million) and matrix entries to 4 decimals.*

1. Introduction

A modern economy is not a set of independent industries. It is a web of suppliers and customers. Manufacturing buys electricity from utilities, utilities buy equipment from manufacturing, and both buy services from finance and transportation. When consumers want more of one good, the demand ripples backward through every industry that supplies it, then through the suppliers of those suppliers, and so on.

The **Leontief Input-Output Model**, developed by Wassily Leontief (Nobel Prize in Economics, 1973), captures this web with a single matrix. The economy is split into n sectors. The **consumption matrix** C records how much each sector needs from every other sector to produce one dollar of output. Each sector's total production x must cover two things:

1. **Intermediate demand** Cx : what the industries themselves use up as inputs, and
2. **Final demand** d : what households, government and exports buy.

This gives the balance equation $x = Cx + d$, a linear system. Linear algebra then answers the key economic questions:

- Can the economy meet *any* demand at all? (Is it **productive**?)
- How much must each sector produce?
- How does a shock in one market spread to sectors that were never directly affected?

In this project we apply the model to a simulated 10-sector national economy.

2. Methodology

2.1 The model

Let $x \in \mathbb{R}^{10}$ be gross production and $d \in \mathbb{R}^{10}$ final demand. Entry c_{ij} of C is the value of sector i 's output that sector j uses to make \$1 of its own output. So column j of C is sector j 's "recipe" of inputs, and $(Cx)_i = \sum_j c_{ij}x_j$ is the total amount of sector i 's output used up by industry. Then

$$x = Cx + d \iff (I - C)x = d.$$

We call $A = I - C$ the **Leontief matrix**. If A is invertible, the unique solution is

$$x = (I - C)^{-1}d = Md,$$

where $M = (I - C)^{-1}$ is the **multiplier matrix** (Leontief inverse).

2.2 Productivity

The economy is **productive** if $(I - C)^{-1}$ exists and has only non-negative entries. Then every non-negative demand d gives a non-negative, physically meaningful production plan $x = Md$.

Why this works: when the spectral radius $\rho(C)$ (the largest $|\lambda|$ over the eigenvalues of C) is less than 1, the **Neumann series** converges:

$$(I - C)^{-1} = I + C + C^2 + C^3 + \dots$$

Each term has a direct economic meaning. d is the demand itself, Cd is the input needed to make d , C^2d is the input needed to make *those* inputs, and so on. Since $C \geq 0$, every term is non-negative, so the sum M is non-negative too.

A simple test for $\rho(C) < 1$ is that every **column sum** of C is below 1. The largest column sum is the matrix norm $\|C\|_1$, and $\rho(C) \leq \|C\|_1$. This test is **sufficient but not necessary**, which matters for this data set (see Part A).

2.3 Computation

All computation uses Python 3 with NumPy, and pandas for labeled tables. The two core operations are:

```
A = np.eye(10) - C          # Leontief matrix
M = np.linalg.inv(A)         # multiplier matrix (needed for Part
A3)
x = np.linalg.solve(A, d)    # equilibrium production (Part B4)
```

Mathematically, `np.linalg.solve(A, d)` and `np.linalg.inv(A) @ d` give the same x . We use `solve` for the answer because it factors A (LU decomposition) and solves directly, which is faster and more accurate than building the inverse and multiplying. After every solve, we check that the solution reproduces the demand:

```
np.allclose(A @ x, d) .
```

2.4 Internal consumption and shocks

- **Internal vs. final (Part B5).** Row i of $x = Cx + d$ splits sector i 's output into $(Cx)_i$, used up by industry, and d_i , delivered to end consumers.
- **Shock propagation (Part C).** The model is linear, so a change in demand Δd changes production by

$$\Delta x = M \Delta d.$$

Column j of M is exactly how much every sector's output moves when final demand for sector j changes by \$1. This explains why sectors whose own demand is unchanged still change their production.

3. Results

3.0 Setup and data

```
In [ ]: import os
import numpy as np
import pandas as pd
from IPython.display import Markdown, display
from data import C, d, SECTORS, SECTOR_NAMES

os.makedirs("outputs", exist_ok=True)
n = len(SECTORS)
MIN, UTL, MAN = SECTORS.index("MIN"), SECTORS.index("UTL"), SECTORS.index("M

def show(df, decimals=4):
    # Display a table with a fixed number of decimals
    return df.style.format(precision=decimals)

C_df = pd.DataFrame(C, index=SECTORS, columns=SECTORS)
d_s = pd.Series(d, index=SECTORS, name="d ($B)")
pd.concat([C_df, d_s.rename("d")], axis=1).to_csv("outputs/consumption_matri
```

Consumption matrix C (row = supplying sector, column = using sector):

```
In [ ]: show(C_df, 2)
```

Final demand d (\$ billions):

```
In [ ]: show(d_s.to_frame().T, 0)
```

3.1 Part A: Feasibility & Structural Analysis

A1. Column sum verification

Economic significance. Column j lists every input sector j buys to make \$1 of output. Its sum is sector j 's **total input cost per \$1 of output**. What is left over, $1 - \sum_i c_{ij}$, is

the **value added** per dollar: the share available for wages, profits and taxes, which is what the sector contributes to the economy beyond the inputs it uses up.

Why each sum should be strictly less than 1. If a sector's column sum is 1 or more, it uses at least \$1 of other sectors' output for every \$1 it produces, so it adds no net value. If every sector were like that, the economy would use up everything it makes just keeping itself running, and nothing would be left for consumers. Column sums below 1 guarantee $\rho(C) \leq \|C\|_1 < 1$. That makes the Neumann series converge, so the economy is productive and can meet any non-negative demand.

```
In [ ]: col_sums = C_df.sum(axis=0)
colsum_table = pd.DataFrame({
    "Sector": [SECTOR_NAMES[s] for s in SECTORS],
    "Column sum": col_sums,
    "Value added per $1": 1 - col_sums,
    "Strictly < 1?": np.where(col_sums < 1, "yes", "NO"),
})
colsum_table.to_csv("outputs/A1_column_sums.csv")
show(colsum_table, 2)
```

```
In [ ]: rho = max(abs(np.linalg.eigvals(C)))
norm1 = np.linalg.norm(C, 1) # = largest column sum
print(f"||C||_1 (largest column sum) = {norm1:.4f}")
print(f"Spectral radius rho(C) = {rho:.4f}")
```

A2. The Leontief matrix $A = I - C$

```
In [ ]: A = np.eye(n) - C
A_df = pd.DataFrame(A, index=SECTORS, columns=SECTORS)
A_df.to_csv("outputs/A2_leontief_matrix.csv")
show(A_df, 2)
```

A3. Multiplier matrix and productivity proof

Entry m_{ij} of $M = (I - C)^{-1}$ is the **total** output, direct and indirect, that sector i must produce for consumers to receive \$1 of sector j 's goods.

```
In [ ]: M = np.linalg.inv(A)
M_df = pd.DataFrame(M, index=SECTORS, columns=SECTORS)
M_df.to_csv("outputs/A3_multiplier_matrix.csv")
show(M_df, 4)
```

```
In [ ]: i, j = np.unravel_index(M.argmax(), M.shape)
print("A @ M == I (inverse is correct):", np.allclose(A @ M, np.eye(n)))
print("All entries of M >= 0", (M >= 0).all())
print(f"Minimum entry", {M.min():.4f} (row {SECTORS[i]}, c
print("All diagonal entries >= 1", (np.diag(M) >= 1).all())
```

3.2 Part B: System Optimization

B4. Equilibrium production

```
In [ ]: x = np.linalg.solve(A, d)
print("Residual check A @ x == d :", np.allclose(A @ x, d))
print("Same as inv(A) @ d      :", np.allclose(x, M @ d))
print("All x >= 0              :", (x >= 0).all())

x_s = pd.Series(x, index=SECTORS)
x_table = pd.DataFrame({
    "Sector": [SECTOR_NAMES[s] for s in SECTORS],
    "Final demand d ($B)": d_s,
    "Production x ($B)": x_s,
    "Production x ($ millions)": (x_s * 1000).round().astype(int),
    "x / d": x_s / d_s,
})
x_table.to_csv("outputs/B4_equilibrium_production.csv")
x_table.style.format({"Final demand d ($B)": "{:.0f}", "Production x ($B)":
    "Production x ($ millions)": "{:,}", "x / d": "{:.2f}"
```

B5. Internal consumption vs. final demand (MAN and UTL)

```
In [ ]: internal = C @ x
rows = []
for s in ["MAN", "UTL"]:
    k = SECTORS.index(s)
    rows.append({
        "Sector": s,
        "Gross output x ($B)": x[k],
        "Internal consumption Cx ($B)": internal[k],
        "To consumers d ($B)": d[k],
        "Internal %": 100 * internal[k] / x[k],
        "Consumers %": 100 * d[k] / x[k],
        "Cx + d = x ?": np.isclose(internal[k] + d[k], x[k]),
    })
split = pd.DataFrame(rows).set_index("Sector")
split.to_csv("outputs/B5_internal_vs_final.csv")
show(split, 3)
```

3.3 Part C: Economic Shock Simulation

C6. Green-energy policy shock

External demand for **MIN** falls to 0, and external demand for **UTL** rises by 40%. All other final demands are unchanged.

```
In [ ]: d_new = d.copy()
d_new[MIN] = 0
d_new[UTL] *= 1.4
```

```
demand_table = pd.DataFrame({"d": d, "d_new": d_new, "Δd": d_new - d}, index=SECTORS)
demand_table.to_csv("outputs/C6_new_demand.csv")
show(demand_table.T, 0)
```

```
In [ ]: x_new = np.linalg.solve(A, d_new)
print("Residual check A @ x_new == d_new:", np.allclose(A @ x_new, d_new))

delta = x_new - x
shock = pd.DataFrame({
    "Own demand changed?": np.where(d_new != d, "yes", "no"),
    "x ($B)": x, "x_new ($B)": x_new, "Δx ($B)": delta, "Δx (%)": 100 * delta / x,
}, index=SECTORS)
shock.to_csv("outputs/C6_shock_results.csv")
show(shock, 3)
```

Why sectors with unchanged demand still move. Since $\Delta x = M \Delta d$ and only Δd_{MIN} and Δd_{UTL} are non-zero,

$$\Delta x_i = m_{i,MIN} \Delta d_{MIN} + m_{i,UTL} \Delta d_{UTL}.$$

Every sector loses business from the Mining collapse and gains business from the Utilities boom. The net effect depends on which link in the supply chain is stronger:

```
In [ ]: dd = d_new - d
decomp = pd.DataFrame({
    "Loss from MIN drop": M[:, MIN] * dd[MIN],
    "Gain from UTL surge": M[:, UTL] * dd[UTL],
}, index=SECTORS)
decomp["Net Δx"] = decomp.sum(axis=1)
assert np.allclose(decomp["Net Δx"], delta)
show(decomp, 3)
```

3.4 Summary of answers

4. Conclusion

```
In [ ]: linkage = pd.DataFrame({
    "Backward linkage (column sum of M)": M.sum(axis=0),
    "Forward linkage (row sum of M)": M.sum(axis=1),
}, index=SECTORS)
show(linkage.sort_values("Forward linkage (row sum of M)", ascending=False).)
```